Taskify: Designing and Developing a Task Management Web Application

Computing Project

REPORT

Edmund Favour Edem Hogan

(P00193367)

September 2026 AfriHUB

Abstract

Paper to-do lists are still common, but they are easy to lose, hard to back up, and awkward to search or sort. Taskify is a task management web application built for this project to give users a simple cloud-backed alternative. A registered user can create an account, log in, add and edit tasks, mark them complete or important, put them in lists, add descriptions and subtasks, search across lists, and send deleted items to a trash can. The application was built with SvelteKit, Svelte 5, TypeScript, and the Qdrant vector database, and it was checked in Chrome, Edge, and Firefox. Features such as due-date reminders, a completion tracker, and location reminders were recorded as later work and were not part of the prototype.

Table of Contents

Chapter 1: Introduction
1.1 The System Developed
1.2 Methodology & Framework Justification
1.3 The Solution that Emerged
1.4 Main Aim & Objectives
1.5 Overview of the Remaining Chapters
Chapter 2: Analysis
2.1 Introduction
2.2 Requirements
2.3 Use Cases
2.4 Architecture
2.5 Conclusion
Chapter 3: Design
3.1 Introduction
3.2 Structural Model
3.3 Behavioural Model
3.4 Conclusion
Chapter 4: Implementation
4.1 Introduction
4.2 Choice of Programming Language
4.3 System Cutover
4.4 Data Migration
4.5 Training
4.6 Conclusion
Chapter 5: Other Project Issues
5.1 Introduction
5.2 Project Management
5.3 Risk Management
5.4 Configuration Management
5.5 Testing
5.6 Conclusion
Chapter 6: Conclusion
References
Appendices

Chapter 1:
Introduction

1.1 The System Developed

This project is centred on the development of Taskify, a task management web application designed to serve all types of users, from students and teachers to business owners and everyday individuals. The primary problem addressed by this system is the inefficiency of managing tasks with the traditional pen-and-paper method. This method creates several drawbacks: paper lists are easily lost or damaged, are difficult to back up, and do not have support for reminders or alarms. Furthermore, they do not sync across devices, offer limited space, and cannot be easily searched, sorted, or reorganised.

As a solution, Taskify provides a digital platform where users can create, edit, and organise tasks that are securely backed up to the cloud and accessible from any device. Specifically, users will be able to make an account, log in and log out, add, edit, and complete tasks, flag tasks as important, delete tasks to trash, include task descriptions, add subtasks, categorise tasks into custom task lists, and search for tasks. Future updates to the application will introduce unique features that will make Taskify stand out from its competitors.

1.2 Methodology & Framework Justification

The project was accomplished using the Agile development methodology. This method was chosen primarily for its iterative approach to software development, as it supports the future intention to incrementally introduce new features or update existing ones. Alongside the Agile framework, object-oriented development principles were also used as a coding technique, for example, within TypeScript modules.

1.3 The Solution that Emerged

The solution that emerged was a web application developed with a modern technical architecture. The front-end of the system was built using the JavaScript framework, SvelteKit (v2.22.0), powered by the Svelte language (v5.0.0) and TypeScript. The user interface (UI) elements were developed using HTML5 and CSS3. For the backend, SvelteKit and TypeScript again handled the server-side logic, and the database was constructed and managed using the Qdrant Vector Database v1.15.0. To ensure a stable final product, the web application was tested for cross-browser compatibility across Google Chrome, Microsoft Edge, and Mozilla Firefox browsers during development.

1.4 Main Aim & Objectives

The primary aim of this project was to design and develop a to-do list web application, Taskify. To achieve this aim, the following objectives were established at the project's outset:

- To perform a detailed analysis of the proposed system's requirements
- To design the system architecture and user interface

- To develop the proposed system and produce a functional working prototype
- To run tests on the working system and fix any bugs and errors discovered
- To document the project lifecycle and produce a final report

1.5 Overview of the Remaining Chapters

The remainder of this report details the work undertaken throughout the project. Chapter 2 presents the analysis of the system, outlining the functional and non-functional requirements, use cases, and the system's architecture, including an initial class diagram. Following this, Chapter 3 details the system design, presenting the final structural and behavioural models. Chapter 4 describes the implementation process, covering the development of the application's core components and training. Chapter 5 discusses other project issues, providing an account of the approach taken to project management, risk management, configuration management and testing. Finally, the report concludes in Chapter 6 with an evaluation of the project's success against its initial aims and objectives, and comments on potential future developments.

Chapter 2:
Analysis

2.1 Introduction

This chapter performs an analysis of the Taskify system. It sets out to identify what the system needs to do to achieve its aim. It does this through first outlining the system's functional and non-functional requirements, the use cases that show what users can do, and the basic architecture plan. This analysis gives the foundation for the system design in the next chapter.

2.2 Requirements

This section outlines the requirements that shaped the development of the Taskify system. These specifications are broken down into two key categories: the functional requirements, which define what the system does, and the non-functional requirements, which define how the system performs.

Functional Requirements

The functional requirements for the Taskify system were prioritised using the MoSCoW technique. This is a method used in project management to help stakeholders understand and agree on the importance of each requirement, particularly within a time-boxed project (Atlassian, 2025). The name itself is an acronym that stands for the four priority categories: **M**ust have, **S**hould have, **C**ould have, and **W**on't have this time (Atlassian, 2025). This framework ensures that a viable product is delivered by distinguishing between non-negotiable needs and desirable features that can be addressed later (Aserg, 2022).

Must Have (M)

These requirements are the core functionality that was implemented in the final prototype:

- The system must allow a user to register for a new account using an email and password, or authenticate using a Google account.
- The system must allow a registered user to log in and log out.
- The system must provide users with the ability to change their username and password.
- The system must allow users to change their profile picture.
- The system must allow users to delete their accounts.
- The system must allow users to add new tasks to the 'Inbox', 'Today', 'Important', or a custom list.
- The system must feature a 'Today' list for tasks to be accomplished on the current day.
- The system must allow users to edit task titles.
- The system must allow users to mark tasks as completed.
- The system must allow users to flag tasks as important.
- The system must allow users to duplicate tasks

- The system must allow users to delete tasks to the trash.
- The system must allow users to restore tasks from the trash.
- The system must allow users to empty the trash (permanently deleting tasks).
- The system must allow users to include detailed descriptions for a task.
- The system must allow users to attach files to a task and preview the file in a modal.
- The system must allow users to add subtasks to a primary task.
- The system must be able to search for tasks across all lists.
- The system must allow users to create, rename, add to, and delete custom task lists.
- Tasks and activities of the same account must sync across devices.

Should Have (S)

These requirements were valued as important for improving the user experience, but were not essential for the initial prototype. They are considered high-priority for future iterations.

- The system should allow users to set due dates and times for tasks.
- The system should allow users to set reminders for tasks.
- The system should remind users of tasks with upcoming due dates or times.
- The system should allow a user to postpone a ‘Today’ task to another day.
- The system should automatically add tasks due on the current day to the ‘Today’ list.
- The system should provide task suggestions from other lists to add to ‘Today’.
- The system should have a completion percentage tracker to show progress within a task list.

Could Have (C)

These requirements were identified as desirable enhancements but were of a lower priority than the 'Should Have' features.

- The system could allow users to set routine reminders for a single task (e.g., every hour).
- The system could allow a user to mark a task as ‘won’t do’.
- The system could allow users to make a task location-based, triggering a reminder upon arriving at and leaving a specific location.
- The system could enable users to include due dates and reminders through natural language interpretation upon task entry.
- The system could allow users to customise when their 'day' begins (e.g., 5 am instead of 12 am) for the purpose of refreshing the 'Today' list.

- The system could allow users to share their task lists with other users and enable collaboration on projects.
- The system could be accessible and interactable without an internet connection.

Won't Have (W)

This represents a specific functionality that was explicitly decided would not be part of the project at this stage.

- The system will not have an installable mobile application for iOS or Android.

Non-functional requirements

Alongside the functional requirements, the following non-functional requirements were established to define the system's operational expectations and the overall user experience.

- Have an intuitive and easy-to-navigate user interface, so a new user can understand how the app works without extensive training.
- Must be fully responsive to enable seamless rendering on various screen sizes, like desktop, tablet, and mobile web browsers.
- Should load and respond to user interactions in under 3 seconds on a standard internet connection.
- User passwords must be securely hashed before they are stored in the database to prevent unauthorised access to user accounts.
- The HTTPS protocol will be used to encrypt data transmitted between the client and the server.
- Must be compatible and render correctly on the latest stable versions of major browsers.
- Must be available 24 hours all the time.

2.3 Use Cases

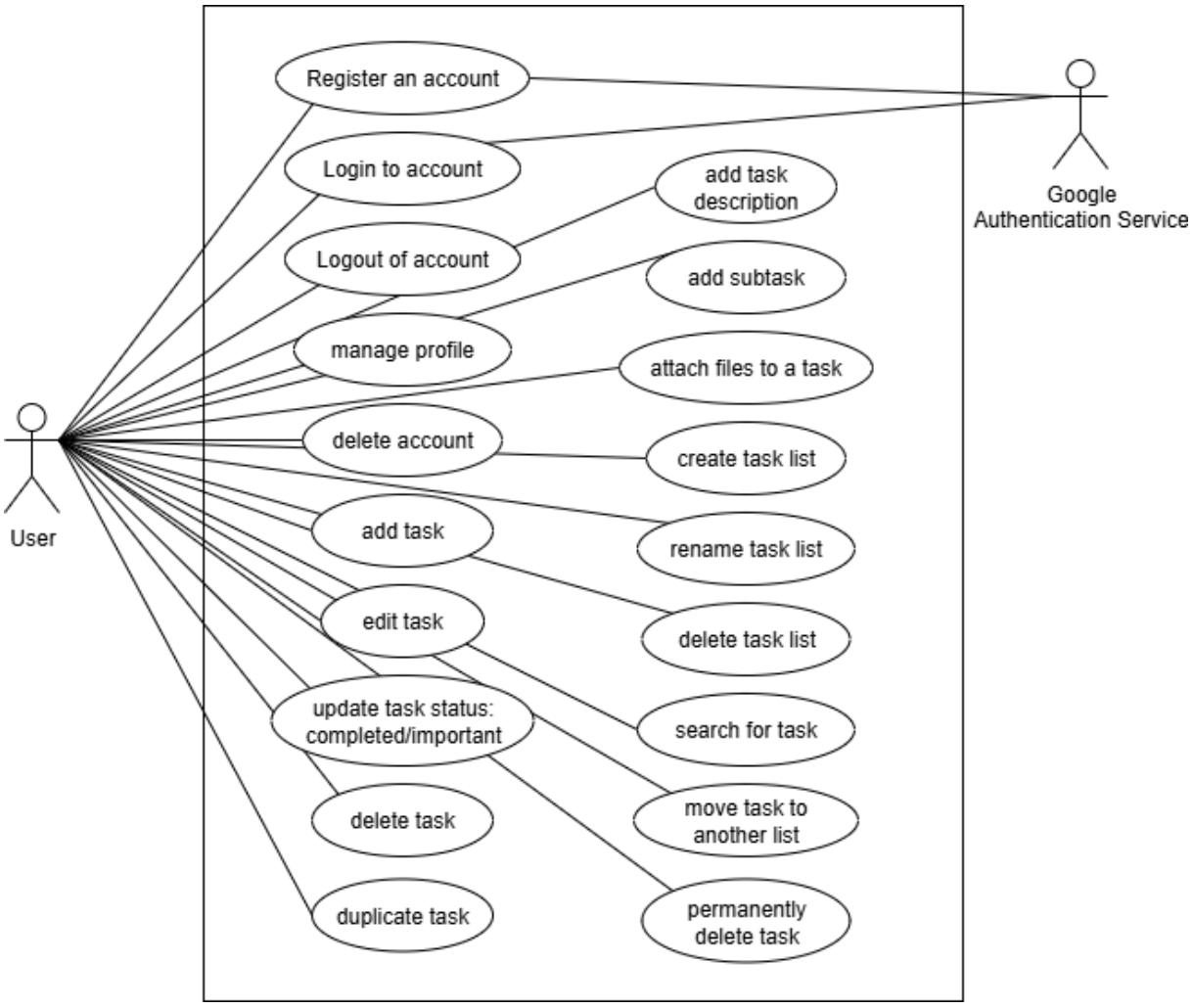


Figure 2.1: Use case diagram

The above diagram (Figure 2.1) defines the use cases for the Taskify system, illustrating the goals that actors can achieve. These use cases are derived from the Must Have functional requirements. The primary actor is the User. The external system actor is the Google Authentication Service. The User can register, log in, log out, manage their profile, create and edit tasks, complete or flag tasks, organise tasks into lists, search, and use the trash. The Google Authentication Service supports sign-in for users who choose that route.

2.4 Architecture

This section presents a high-level overview of the proposed system architecture for Taskify. It outlines the technologies chosen for development.

System Architecture Description

The system is designed with a modern architecture where the front-end will be developed using the JavaScript framework, SvelteKit version 2.22.0, using the Svelte language version 5.0.0, and TypeScript. Standard User Interface (UI) elements will be built using HTML and

CSS. For the backend, SvelteKit and TypeScript will also handle the server-side logic. The database will be constructed and managed using the Qdrant Vector Database version 1.15.0. To ensure cross-browser compatibility, the web application will be tested across Google Chrome, Microsoft Edge, and Mozilla Firefox.

Initial Class Diagram

Following the architecture description, the initial class diagram for the system is shown below:

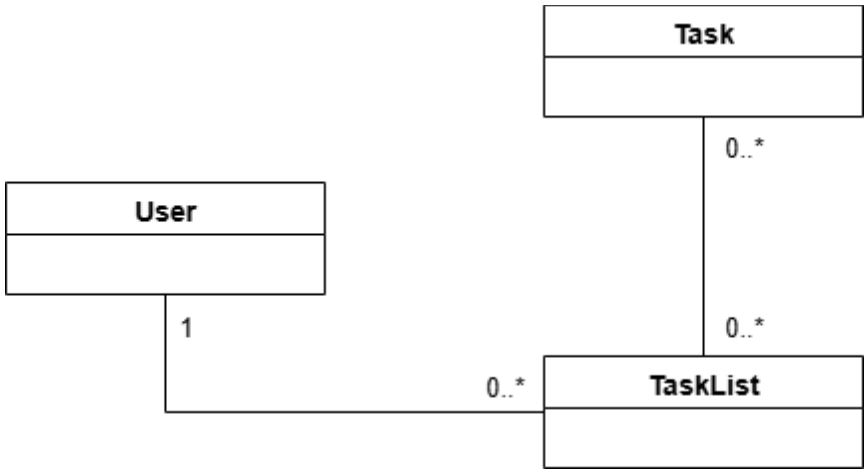


Figure 2.2: Initial Class Diagram

The above diagram (Figure 2.2) presents the static structure of the system at a high level. This model identifies the key classes of the system—User, TaskList, and Task—and the basic relationships between them. At this stage, the diagram focuses on the associations and multiplicities, showing that a User can own multiple TaskLists, and that each TaskList can contain multiple Tasks, and a Task can belong to multiple TaskLists, reflecting a many-to-many relationship. A more detailed version of this diagram, including attributes and methods and more specific relationships, is presented in the Design chapter.

2.5 Conclusion

The analysis of the Taskify system has been detailed. It defined the functional and non-functional requirements using the MoSCoW technique, identified the use cases in a use case diagram, and discussed the system's architecture, including modelling an initial class diagram. These analysis specifications will now be used as a guide to inform the detailed design in Chapter 3.

Chapter 3:
Design

3.1 Introduction

This chapter discusses the design for the Taskify system. It creates a technical blueprint for how the app was put together and would operate based on the analysis in Chapter 2. The design is broken down into two main sections. The first one is the Structural Model, which shows the different classes and how they are related. The second one is the Behavioural Model, which shows how those classes would interact with each other to get things done.

3.2 Structural Model

This section illustrates the final structural model for the Taskify system in the detailed class diagram below (Figure 3.1). It shows all the main classes, their attributes and methods, and how they relate to each other.

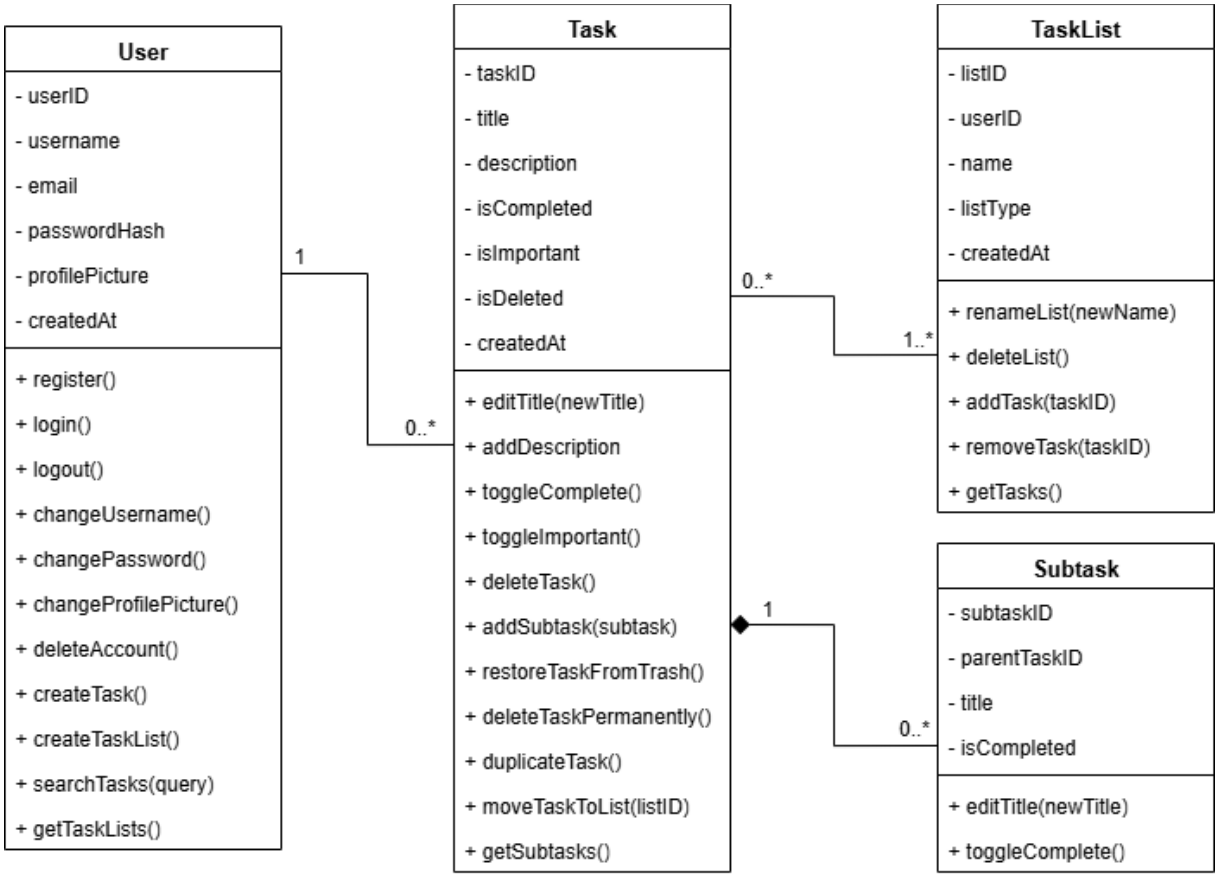


Figure 3.1: Final Class Diagram

The Classes

- User: This class holds all the information for a user's account, like their **username** and **passwordHash**. It also contains all the methods for managing the account, such as **register()** and **login()**, as well as methods for creating tasks and lists, and for searching.
- TaskList: This class represents a list that holds tasks. It has a name and a **listType** attribute, which is used to tell if the list is a custom one made by the user or a smart list like 'Today' or 'Important'.

- Task: This is the core class of the application. It stores everything about a single to-do item, including its title, description, and whether it is **isCompleted** or **isImportant**. The methods here cover all the actions you can do with a task, like **toggleComplete()**, **deleteTask()**, and **moveTaskToList()**.
- Subtask: This is a simpler version of a task that is always attached to a parent Task.

Relationships Between the Classes

The connections between the classes define how the system works:

- One **User** can own zero or many (0..*) **Tasks**, and a **Task** belongs to only one (1) **User**.
- A **TaskList** can hold zero or more (0..*) **Tasks**, and a **Task** can belong to one or more (1..*) **TaskLists,** e.g. a custom list and the 'Today' list. This is the key relationship that lets a single task appear in multiple lists at the same time.
- Finally, the relationship between **Task** and **Subtask** is a composition, which is shown by the filled-in diamond. This means a **Subtask** is a part of a **Task** and cannot exist by itself. If a **Task** is deleted, all of its **Subtasks** are deleted along with it.
- A **Task** can have zero or more (0..*) **SubTasks**, and a **SubTask** can only belong to one **Task** (1).

3.3 Behavioural Model

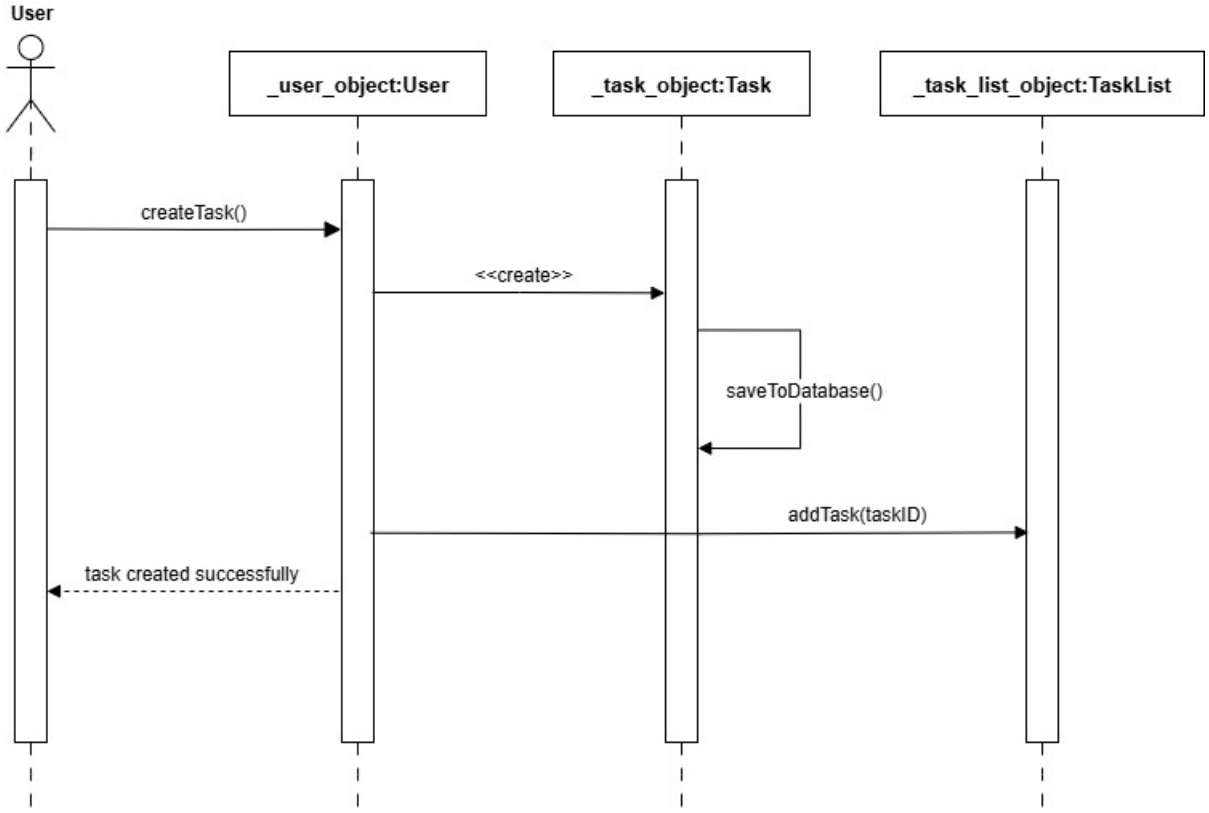


Figure 3.2: Sequence Diagram for Creating a Task

This section shows the behavioural model for the Taskify system. A sequence diagram is used to show how different objects in the system interact with each other step-by-step to complete a process.

The diagram above (Figure 3.2) shows the sequence for the 'Create Task' use case. It shows how the User actor starts the process, and how the $:\mathrm{User}$, $:\mathrm{Task}$, and $:\mathrm{TaskList}$ objects work together to create a new task, save it, and add it to the correct list.

3.4 Conclusion

This chapter has laid out the detailed design of the Taskify system. It first presented the structural model, which consisted of the final class diagram showing all the classes, the attributes and methods, and their relationships. Then, it introduced the Behavioural Model with a sequence diagram showing how objects interact to create a task. This design blueprint was then used to guide the actual development of the application. More on this in the next chapter, which expands on how the application was built and the plans for launching it.

Chapter 4:
Implementation

4.1 Introduction

This chapter goes into detail on how the Taskify web app was developed. It takes the design blueprint from Chapter 3 and describes the process of turning it into a real, working application for this project. After that, it will cover other important topics, like the plan for launching the app, data migration, and user training.

4.2 Choice of Programming Language

A set of programming languages and frameworks was chosen to build the app's front-end, back-end, and database. They were selected for their ability to bring to life the project idea in the way it needed. This section will justify why those selections were chosen, beginning with the front-end.

Front-end

The front-end of Taskify was built using SvelteKit (v2.22.0), powered by Svelte 5 (v5.0.0) and TypeScript. HTML5 and CSS3 were used for the structure and styling.

SvelteKit was chosen because it is a lightweight framework that emphasises speed. Unlike other frameworks that use a virtual DOM at runtime, SvelteKit compiles components into highly optimised vanilla JavaScript at build-time and updates the real DOM directly, making the app feel instant for the user. Svelte also has much simpler and more straightforward syntax to code with compared to other front-end frameworks.

TypeScript is a superset of JavaScript that adds static types. It lets you define types for variables, functions, objects, etc. TypeScript was integrated to provide compile-time type checking. Adding static type safety to JavaScript allowed me to catch errors and bugs while writing the code (compile-time and build-time), rather than discovering them later at runtime, because it makes sure I'm using the right kind of data in the right places.

Back-end

SvelteKit is a full-stack-capable framework, which means it could also handle the server-side logic of Taskify. This was a major advantage because it allowed the entire application to be written in a single language, TypeScript. This made development much smoother because I didn't have to switch contexts between different languages and frameworks. This kept the project structure clean and simple, and communication between the client and server seamless.

Database

Qdrant is an open-source vector database and vector search engine written in the Rust programming language. It efficiently stores, indexes, and searches high-dimensional vectors (embeddings) for similarity search, semantic search, recommendation systems, RAG, etc. Qdrant v1.15.0 was used to store Taskify's data. While SQL databases are common for to-do

lists, Qdrant enables embedding-based search. This means tasks can be searched based on their meaning, not just exact keyword matches, displaying in the search results tasks that are related in meaning to the search keyword. Additionally, its payload feature acts as a flexible key-value store, which makes filtering tasks fast (like showing only ‘Important’ or ‘Completed’ items).

4.3 System Cutover

The system cutover from the development architecture to the implementation would implement the immediate/big bang cutover strategy and the parallel cutover strategy because users migrating to the system are divided into two types: users transitioning from the pen-and-paper method and users from other task management systems like Todoist or Google Tasks. Users from pen and paper will use the immediate cutover strategy, where they immediately discontinue use of their old method and adopt the new system by creating an account on Taskify and continuing to add and complete tasks as usual.

Users from other systems, like Google Tasks, will transition with the parallel cutover strategy to help them properly migrate all their data from the old system to Taskify without disrupting their productivity flow. For example, a user from Google Tasks, after onboarding on Taskify, will use Taskify along with their old system side-by-side. This is the "parallel running" phase. They may put new tasks in Taskify while still checking their old tasks in the old system. This method will involve another strategy, the user-driven cutover. This means the user themselves decides when to fully "cut over." This happens when they feel comfortable enough with Taskify and have moved all their important data across. At this point, they stop using Todoist and use Taskify exclusively.

Deployment of the application to the public internet for users to access will be done by launching it with a reliable hosting service. The chosen hosting service to launch Taskify will be Vercel. Vercel is a cloud platform built specifically for modern web frameworks like SvelteKit. It makes launching a website very easy because it connects directly to a developer's GitHub account and automatically builds and deploys the application every time new code is pushed, making the process of getting the site online and updating it fast and simple.

In conclusion, the system cutover plan for Taskify is a user-relative adoption, supported by a data migration plan to make the switch easier.

4.4 Data Migration

To migrate all important user data from their old systems to the new system, Taskify, different strategies can be employed. For users transitioning from pen and paper, they would simply register on Taskify and transfer all uncompleted tasks from their paper list to a Taskify task list manually. For users transitioning from digital systems, the data migration plan currently

available to them will be to use Taskify and their old system in parallel. They could do this by adding new tasks to Taskify and only tracking ongoing tasks on their previous system. After completing the ongoing tasks, the user can now fully use Taskify exclusively.

Another method that could be employed and would make data migration much easier for users would be to use a CSV (Comma-Separated Values) file import/export. The user would first go to their old app (like Todoist) and use its "Export" feature to save all their tasks as a CSV file. Then, in Taskify, there would be an "Import from CSV" feature. The user would upload their file, and the system would then read the CSV file, and for each row, it would create a new task in the user's Taskify account.

However, this approach is currently unavailable. Furthermore, the development of such a feature is tricky because the CSV template format of the other apps will differ. Taskify will have to intelligently import data from those templates to match its own.

Therefore, the data migration plan for Taskify is (1) Immediate transfer of all uncompleted tasks from paper lists to Taskify. (2) Adding new tasks only to Taskify and completing ongoing tasks on the old system until full adoption of Taskify.

4.5 Training

Since Taskify is a web application designed for the general public, there were no formal training sessions. Instead, the training strategy relies on the system’s simple and intuitive design, an onboarding guide, and helpful tips across the app. The user interface was built using familiar patterns and layouts found in other similar applications. This allows users to quickly learn how to use the system simply by exploring it. For every first-time user, an onboarding guide takes them on an overview highlighting the major sections of the app, and describing what they are and how to use them. The guide can be replayed again later if the user ever needs it. Pop-up tooltips were also included in the interface to provide users with additional information on certain features, such as buttons and links.

4.6 Conclusion

This chapter has described the practical steps taken to build and launch Taskify. It first explained the technologies used to develop the project—the programming languages, frameworks, and the database system used—and provided justifications for why they were chosen. Then, it proceeded to outline the plan for getting users onto the system and moving them from any previous systems—the system cutover and data migration strategies. Finally, it discussed the user training plan needed for the system, explaining the simple and intuitive fashion in which the system was designed, so users could train themselves. With the system built and the launch plan ready, the next chapter will discuss how the project was managed and tested.

Chapter 5:
Other Project Issues

5.1 Introduction

This chapter provides an account of the management and quality-assurance processes used during the project's development. While the previous chapters focused on the technical construction of the application, this chapter focuses on how the project work was organised and controlled. It details the approach taken to project management, including work scheduling and methodology. It also outlines how potential risks were managed, how the software versions were organised through configuration management, and finally, the testing strategy used to ensure the system functions correctly.

5.2 Project Management

The Taskify project was managed using the Agile development methodology. This approach broke the work into distinct phases, as shown in the Work Breakdown Structure below. This iterative structure provided flexibility, allowing complex tasks like the backend logic and database setup to be addressed step-by-step. Microsoft Excel was used to produce the Gantt chart as well as schedule activities and track progress. This tool provided a clear visual representation of the project timeline and helped in monitoring deadlines.

Project Timeline and Adherence

The project generally followed the planned dates set out in the proposal. However, there were extensions to some phases due to a lack of understanding while writing the report. So, I had to consult with my supervisor for assistance. The project was originally scheduled to conclude on 7th January 2026, but it was finally completed on 25th January 2026. This adjustment was necessary to ensure the quality of the application and documentation phase.

Work Breakdown Structure and Gantt Chart

The project was undertaken across six distinct phases within a timeline of 4 months (119 days).

Tasks	Start Date	Days	Completion Date
Phase 1: Project Initiation and Planning	10-Sep-25	7	17-Sep-25
Phase 2: Analysis and Requirements Gathering	17-Sep-25	21	08-Oct-25
Phase 3: Designing the system	08-Oct-25	14	22-Oct-25
Phase 4: Developing the system	22-Oct-25	42	03-Dec-25
Phase 5: Testing the system and fixing bugs	03-Dec-25	21	24-Dec-25
Phase 6: Final Documentation and Handover	24-Dec-25	14	07-Jan-26

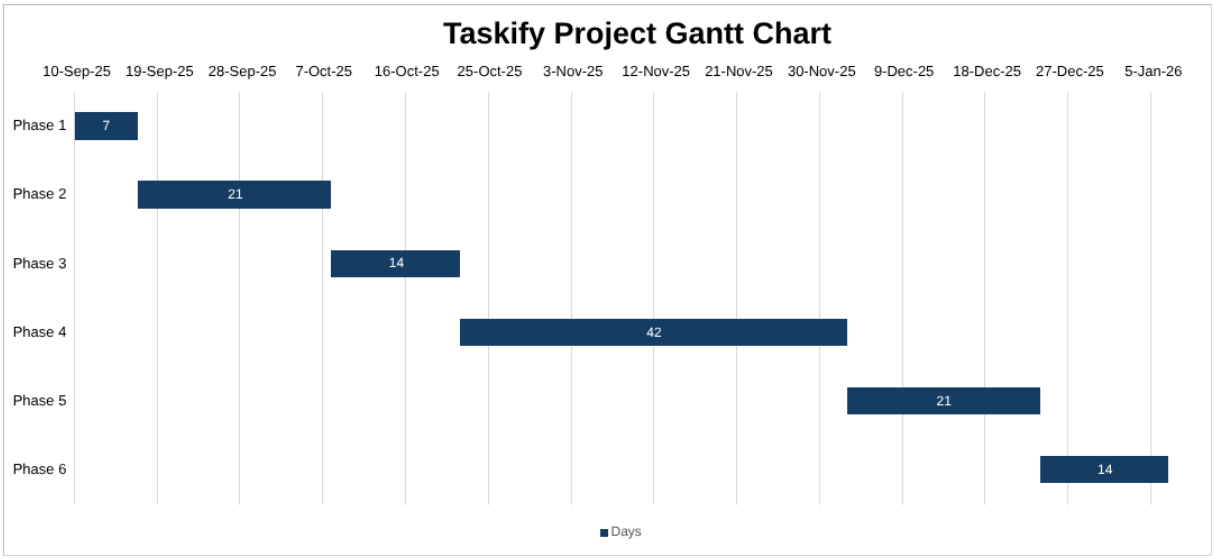


Fig. 1: Gantt chart for the project

5.3 Risk Management

Risk management involves the identification of problems that could threaten the project's success and putting plans in place to handle them. For this project, I identified three main risks that threatened the completion of the project. I assessed how likely they were to happen (Likelihood) and how bad it would be if they did (Impact). The table below outlines these risks and the actions or contingencies I put in place to mitigate them.

Risk	Impact	Likelihood	Severity	Tracker	Action/Contingency
Computer crashing / Data loss	10	5	50	Edmund	(A) Set up the Google Drive for Desktop software to automatically back up all project files and documents to the cloud. I also used GitHub to push and back up my code versions regularly.
Including too many features	8	8	64	Edmund	(A) Prioritised the development of only the primary features needed to execute the basic functions of a to-do list app (Must Have). I set aside more advanced features (Should/Could Have) for future updates.
Insufficient storage capacity in the database	9	10	90	Edmund	(C) Switched from using Redis database to Qdrant database because Redis has a storage limit of 25 MB for free users, and Qdrant has 4 GB.

The first risk, computer crashing/data loss, is given a high impact rating (10/10) because if my computer crashes and I lose all my data without a backup, I would lose the entire project source code and reports and have to redo all of it. This would affect my ability to keep to the timeframe and meet the deadline. Its likelihood rating is medium (5/10) because while modern laptops are generally reliable, hardware/software failures and even theft happen often enough to be a real possibility during a long project.

The second risk, including too many features, is rated high on Impact (8/10) because if such a risk occurs, it could cause time slippage. "Must Have" features (like flagging tasks as important) may be rushed to make time for the fancy ones, leading to bugs and reduced quality of the core features. Furthermore, such unnecessary additions could reduce my time for writing the report or fixing critical errors before the project deadline. Notably, the risk has an equally high likelihood (8/10) due to the fact that I can get quite excited about projects I'm developing and will constantly brainstorm cool new features to add. Luckily, the risk was well mitigated thanks to the containment plan (action) implemented.

The final risk, insufficient storage capacity in the database, is given a high impact rating of 9/10 because running out of space on an active web application is a terrible situation. If the storage does fill up, the application will stop allowing users to create new tasks or save changes. This breaks the app's core functionality, rendering it unusable. The likelihood of occurrence (10/10) is also high because Redis offers only 25MB on its free tier. Therefore, space can quickly be exhausted as vector data takes up more storage than simple text, especially when embedding-based search is included.

5.4 Configuration Management

Configuration management is the process of organising, managing, and tracking changes of all project assets. In a development workflow where files and code change constantly, proper configuration management is needed because you can easily overwrite important progress, misplace supporting files, or need to fall back to a previous stable version. This section outlines how configuration management was handled for Taskify.

For this project's configuration management, a lightweight approach was adopted. This involved a clear folder structure, automated cloud backup with built-in version history via Google Drive for documentation, and Git/GitHub for source code version control and backup. This approach was chosen because the project is a single-developer student build: enterprise configuration servers would add cost and setup time without changing the need to back up files, see older versions, and restore a last-good copy. Every file was still backed up, changes could be tracked, and previous states could be restored without keeping clutter such as Report_v1 and Report_v2.

The items placed under configuration control included all artefacts produced during the project lifecycle:

- **Documentation:** Project proposal, project report, presentation slides, and demonstration report.
- **Project Management:** Microsoft Excel workbook containing the Work Breakdown Structure, Gantt chart, and timeline tracker.
- **Design Models:** Visual diagram files (Use Case diagram, Class diagrams, and Sequence diagram).
- **Media and Assets:** Screenshots of the UI, diagrams, and exported charts.
- **Source Code:** All application files, including SvelteKit routes, Svelte 5 components, TypeScript logic, CSS stylesheets, configuration files, and database connection scripts.

The supporting documents and files of the project were organised into a single main directory called TASKIFY PROJECT to keep all assets centralised. The application itself was stored in a separate folder to prevent it from syncing to Google Drive and instead sync to GitHub. The folder structure is laid out below:

TASKIFY PROJECT/	(Main Project Folder)
DOCUMENTATION/	(Reports, Presentation, and Project Management)
Project Report.docx	
Demonstration Report.docx	
Presentation Slides.pptx	
Project Management.xlsx	
DIAGRAMS/	(Files and images of the UML diagrams)
use case diagram.drawio	
initial class diagram.drawio	
final class diagram.drawio	
sequence diagram.drawio	
IMAGES/	(UI and source code screenshots, diagram exports, and the Gantt chart)
TASKIFY APP/	(SvelteKit application files)

- **Documentation Versioning:** Rather than manually creating separate draft files, versioning was managed through Google Drive's automated **Version History**. Every significant edit to the report or presentation was saved with a timestamp, allowing any previous state to be restored if needed.

- **Code Versioning:** The `SOURCE_CODE` directory operated as a local Git repository linked to a remote GitHub repository. After every development session, changes were committed and pushed to the `main` branch. This maintained a complete historical log of all code changes and provided a safe rollback point if new code caused issues.

All configuration items were stored and edited on an HP EliteBook running Windows 11. The application stack was SvelteKit 2.22.0, Svelte 5.0.0, TypeScript, HTML5, CSS3, and Qdrant 1.15.0. Google Drive for Desktop synced the documents. Git and GitHub held the source code. The live site was planned for Vercel. Browser checks used the current stable builds of Google Chrome, Microsoft Edge, and Mozilla Firefox. A full list is in Appendix C.

5.5 Testing

Testing was used to check that the working prototype did what the Must Have requirements asked for, and to find defects before the report was finished. The work sat in Phase 5 of the plan (21 days) and followed the black-box, unit, and integration checks named in the work breakdown.

Black-box tests were run as a user of the live interface. The cases covered account registration with email and password, Google sign-in, login and logout, changing a username or password, changing a profile picture, and deleting an account. On the task side they covered adding a task to Inbox, Today, Important, or a custom list, editing a title, marking a task complete or important, duplicating a task, moving a task to trash, restoring it, emptying trash, adding a description, attaching a file, adding subtasks, creating and renaming lists, and searching across lists. Each case passed when the screen showed the expected result and the data was still there after a refresh.

Unit and integration checks sat behind the interface. Authentication routes were checked for valid and invalid credentials. Task and list routes were checked for create, update, complete, flag, trash, restore, and delete. Search was checked for both a matching title and a related phrase, because Qdrant stores embeddings as well as the task payload. Failures were logged, fixed in the TypeScript modules, and run again.

The same checks were repeated in the latest stable builds of Google Chrome, Microsoft Edge, and Mozilla Firefox, as set out in Chapter 1. Layout and the main actions were also tried at desktop and narrower widths, because a non-functional requirement was that the interface stay usable on different screen sizes.

Defects found during this pass were treated as blocking if they broke a Must Have action. Examples included a stale list after a delete, a search result that did not open the right task, and a session that did not clear on logout. Those were fixed before the prototype was treated

as complete. Should Have and Could Have items (due dates, reminders, the completion tracker, location reminders) were not in the build, so they were not tested.

The test set was written by the developer and run by hand. That is enough to show the prototype works for the required cases, but it is not an independent test team and there is no automated suite in continuous integration. Those would be the next step if the application were taken further. The case-by-case record is Table 5.1 below, and is repeated as Appendix B.

Table 5.1: Test record for Must Have features

ID	Case	Type	Result
T01	Register with email and password	Black-box	Pass
T02	Sign in with Google	Black-box	Pass
T03	Log in and log out	Black-box	Pass
T04	Change username and password	Black-box	Pass
T05	Change profile picture	Black-box	Pass
T06	Delete account	Black-box	Pass
T07	Add a task to Inbox, Today, Important, or a custom list	Black-box	Pass
T08	Today list shows current-day tasks	Black-box	Pass
T09	Edit a task title	Black-box	Pass
T10	Mark a task complete	Black-box	Pass
T11	Flag a task as important	Black-box	Pass
T12	Duplicate a task	Black-box	Pass
T13	Delete a task to trash	Black-box	Pass
T14	Restore a task from trash	Black-box	Pass
T15	Empty the trash	Black-box	Pass
T16	Add a task description	Black-box	Pass
T17	Attach a file and preview it	Black-box	Pass
T18	Add subtasks	Black-box	Pass
T19	Search across lists (title and related phrase)	Black-box / integration	Pass
T20	Create, rename, add to, and delete a custom list	Black-box	Pass
T21	Auth routes reject invalid credentials	Unit / integration	Pass
T22	Task and list API create, update, trash, restore, delete	Unit / integration	Pass
T23	Chrome, Edge, and Firefox: main actions	Compatibility	Pass
T24	Layout usable at desktop and narrower widths	Compatibility	Pass
T25	Due dates, reminders, completion tracker, location reminders	Out of scope	Not tested

5.6 Conclusion

This chapter described how the Taskify project was organised and controlled. It outlined the Agile approach, the work breakdown, and the timeline, including the slip from 7 January 2026 to 25 January 2026. It then set out the three main risks and how they were handled: extra features, data loss, and running out of database space. Configuration was kept light, with documents on Google Drive and code on GitHub. Testing was used to check the Must Have features of the working prototype. Chapter 6 now closes the report by checking the work against the original aim and objectives.

Chapter 6:
Conclusion

This chapter closes the report. It checks the finished work against the aim and objectives in Chapter 1, states what was left out, and notes possible later work.

The aim was to design and develop Taskify, a to-do list web application. That aim was met. A working web application is in place. A user can register, log in, manage tasks and lists, search, and keep data in the cloud. The stack described in Chapters 1 and 4 — SvelteKit, Svelte 5, TypeScript, HTML, CSS, and Qdrant v1.15.0 — is the stack that was used.

The five objectives were met in the main. The system was analysed, with functional and non-functional requirements, use cases, and an initial class diagram (Chapter 2). It was designed, with a final class diagram and a create-task sequence diagram (Chapter 3). A working prototype was built and a launch, migration, and training plan was written (Chapter 4). Tests were run on the Must Have features and defects were fixed (Chapter 5). This report documents that path.

Two limits should be stated plainly. First, the extras named in Chapter 1 — a completion tracker, postponing tasks, routine reminders, and location reminders — were left as Should Have or Could Have items and were not built. Second, testing was developer-run and manual. Both are honest gaps, not hidden ones.

Later work follows from those gaps. The Should Have set (due dates, reminders, postpone, the completion tracker) is the first increment. After that come the Could Have items, a CSV import for users leaving another app, and an automated test suite. The same Agile split used on this project is a fit for that sequence.

In summary, the core product is in place and the original aim was met. What remains is to grow the later features when there is more time, and to tighten testing if the application is taken beyond a student prototype.

References

ASERG (2022) Moscow prioritisation method. Available at: https://aserg.codeberg.page/shu-dev-process-legacy/modelling-analysis/level-5/moscow-prioritisation-method/ (Accessed: 30 September 2026).

Atlassian (2025) MoSCoW method. Available at: https://www.atlassian.com/agile/project-management/moscow (Accessed: 30 September 2026).

Qdrant (2025) Qdrant vector database documentation. Available at: https://qdrant.tech/documentation/ (Accessed: 30 September 2026).

Svelte (2025) Svelte and SvelteKit documentation. Available at: https://svelte.dev/docs (Accessed: 30 September 2026).

Vercel (2025) Vercel documentation. Available at: https://vercel.com/docs (Accessed: 30 September 2026).

Appendices

Appendix A — Work breakdown

This appendix is the six-phase work breakdown used to plan and track the project. The same list sits behind the Gantt chart in Chapter 5.

Phase 1: Project Initiation and Planning (7 days)

- Developing the initial project idea
- Securing approval for the project idea from my supervisor
- Drafting the project proposal
- Securing supervisor approval for the proposal

Phase 2: Analysis and Requirements Gathering (21 days)

- Conducting a review of existing solutions to inform the project's direction
- Gathering the system's functional and non-functional requirements
- Modelling a Use Case diagram to illustrate the interactions between users and the system
- Outlining the system's architecture
- Modelling an initial class diagram to define the system's static structure
- Securing supervisor approval for the completed analysis and requirements documentation

Phase 3: Designing the system (14 days)

- Modelling the final, detailed class diagram, supported by an appropriate narrative
- Modelling a sequence diagram to describe the behaviour of a key system operation.

Phase 4: Implementing the system (42 days)

- Setting up the database schema using Qdrant Vector Store.
- Implementing the backend server-side logic, including user authentication and task management APIs.
- Developing the front-end user interface and connecting it to the backend services.

Phase 5: Testing the system and fixing bugs (21 days)

- Performing diverse black-box tests to verify that the application's features work as expected from a user's perspective.
- Developing and documenting unit and integration test scripts to validate individual components and their interactions.
- Identifying, logging, and systematically fixing any bugs, errors, or defects discovered during testing.

Phase 6: Final Documentation and Handover (14 days)

- Writing the final project report by fleshing out each chapter and arranging a comprehensive appendix.
- Ensuring the final source code is well-commented and clean.
- Preparing and delivering the final project presentation and demonstration.
- Printing and submitting all project deliverables, including the proposal document, final report, demonstration report, and presentation slides.

Appendix B — Test record

Table B.1 repeats the test record from Chapter 5. Pass means the case was run on the prototype and the expected result was seen after any defect found in that pass had been fixed. Not tested means the feature was not in the build.

Table B.1: Test record for Must Have features

ID	Case	Type	Result
T01	Register with email and password	Black-box	Pass
T02	Sign in with Google	Black-box	Pass
T03	Log in and log out	Black-box	Pass
T04	Change username and password	Black-box	Pass
T05	Change profile picture	Black-box	Pass
T06	Delete account	Black-box	Pass
T07	Add a task to Inbox, Today, Important, or a custom list	Black-box	Pass
T08	Today list shows current-day tasks	Black-box	Pass
T09	Edit a task title	Black-box	Pass
T10	Mark a task complete	Black-box	Pass
T11	Flag a task as important	Black-box	Pass
T12	Duplicate a task	Black-box	Pass
T13	Delete a task to trash	Black-box	Pass
T14	Restore a task from trash	Black-box	Pass
T15	Empty the trash	Black-box	Pass
T16	Add a task description	Black-box	Pass
T17	Attach a file and preview it	Black-box	Pass
T18	Add subtasks	Black-box	Pass
T19	Search across lists (title and related phrase)	Black-box / integration	Pass
T20	Create, rename, add to, and delete a custom list	Black-box	Pass
T21	Auth routes reject invalid credentials	Unit / integration	Pass
T22	Task and list API create, update, trash, restore, delete	Unit / integration	Pass
T23	Chrome, Edge, and Firefox: main actions	Compatibility	Pass
T24	Layout usable at desktop and narrower widths	Compatibility	Pass
T25	Due dates, reminders, completion tracker, location reminders	Out of scope	Not tested

Appendix C — Hardware and software

The project was developed on one machine. Documents lived on Google Drive. Source code lived on GitHub. The planned public host was Vercel.

Item	Detail		

Computer	HP EliteBook, Windows 11		
Documents	Google Drive for Desktop, Version History		
Source control	Git, remote on GitHub, main branch		
Front-end	SvelteKit 2.22.0, Svelte 5.0.0, TypeScript, HTML5, CSS3		
Back-end	SvelteKit server routes in TypeScript		
Database	Qdrant 1.15.0		
Host (planned)	Vercel, linked to GitHub		
Browsers checked	Google Chrome, Microsoft Edge, Mozilla Firefox (latest stable)		
Diagrams	draw.io exports for use case, class, and sequence diagrams		
Planning	Microsoft Excel: WBS, Gantt chart, timeline		

Table C.1: Hardware and software used on the project